C.2.3. AI Scientist Team Code Review

Temperature scaling

In our review of the paper, we noted that it lacked experiments involving temperature scaling. Upon inspecting the generated code, we found that the AI Scientist had implemented temperature scaling, as can be seen in fig. 8, but never actually used it.

During the paper writing stage, the AI Scientist had access to a set of generated experiment code and its initial plans before generating the code. As a result, it is likely that the paper was influenced by these plans and code, which included temperature scaling, but the AI Scientist failed to realize that the experiments using temperature scaling were never actually conducted.

```
class TemperatureScaling(nn.Module):
    def __init__(self):
        super(TemperatureScaling, self).__init__()
        self.temperature = nn.Parameter(torch.ones(1))

    def forward(self, logits):
        return logits / self.temperature
```

Figure 8 | Temperature scaling implementation.

Dataset class

We found that the initial implementation of the dataset class lacked an option for symmetric/asymmetric noise distribution, even though it was part of the initial plan. The AI Scientist recognized this mistake and later implemented the correct version, as shown in fig. 9.

In the main paper, the AI Scientist wrote: "Assymetric Noise: Labels are flipped to specific incorrect classes based on a predefined confusion matrix, simulating more realistic mislabeling." The asymmetric noise implementation in the generated code always maps class i to class (i+1) % NUM CLASSES. While this is a valid approach, it is worth noting that there are other ways to implement asymmetric noise.

```
# Data preparation with noise injection
class NoisyDataset(Dataset):
    def __init__(self, dataset, noise_rate=0.2):
        self.dataset = dataset
        self.noise_rate = noise_rate
        self.noisy_labels = self._inject_noise()

    def _inject_noise(self):
        labels = np.array([y for _, y in self.dataset])
        mask = np.random.rand(len(labels)) < self.noise_rate
        noisy_labels = labels.copy()
        noisy_labels[mask] = np.random.randint(0, NUM_CLASSES, mask.sum())
        return torch.LongTensor(noisy_labels)

    def __getitem__(self, index):
        image, _ = self.dataset[index]
        return image, self.noisy_labels[index]

    def __len__(self):
        return len(self.dataset)
```

```
class NoisyDataset(Dataset):
    def __init__(self, dataset, noise_type="symmetric", noise_rate=0.2):
        self.dataset = dataset
        self.noise_rate = noise_rate
        self.noise_type = noise_type
        self.noisy_labels = self._inject_noise()

    def _inject_noise(self):
        labels = np.array([y for _, y in self.dataset])
        if self.noise_type == "symmetric":
            mask = np.random.rand(len(labels)) < self.noise_rate
            noisy_labels = labels.copy()
            noisy_labels[mask] = np.random.randint(0, NUM_CLASSES, mask.sum())
        else:  # asymmetric noise
            noisy_labels = labels.copy()
            for i in range(NUM_CLASSES):
                mask = (labels == i) & (np.random.rand(len(labels)) < self.noise_rate)
                noisy_labels[mask] = (i + 1) % NUM_CLASSES
        return torch.LongTensor(noisy_labels)

    def __getitem__(self, index):
        image, _ = self.dataset[index]
        return image, self.noisy_labels[index]

    def __len__(self):
        return len(self.dataset)
```

Figure 9 | Noisy dataset class implementation.

Evaluation function

The evaluation function used to compute the Expected Calibration Error is shown in fig. 10. We manually created test cases and used the MulticlassCalibrationError function with norm='l1' from torchmetrics as the ground truth. Since the MulticlassCalibrationError function expects probability inputs, we omitted the softmax operation in the first line to align with the implementation details.